CrewAI Project

Full Implementation Guide

CrewAI v0.80+ Python 3.10-3.13 Production-Ready

Reference document for building a production-ready CrewAI project from scratch.
Covers agents, tasks, tools, flows, memory, testing, and deployment.

Generated: June 2026

CrewAI Project -- Full Implementation Guide

> *Reference document for building a production-ready CrewAI project from scratch.*

Table of Contents

---

1. What is CrewAI

CrewAI is a Python framework for orchestrating **role-based autonomous AI agents** that collaborate to complete complex tasks. Key design pillars:

Concept	Description
Agent	An AI entity with a role, goal, backstory, and a set of tools
Task	A discrete unit of work assigned to an agent
Crew	A group of agents + tasks wired together with a process
Tool	A callable capability an agent can invoke (web search, file I/O, APIs, etc.)
Flow	A higher-level state-machine that orchestrates one or many Crews

Two execution **Process** modes: - `sequential` -- tasks run one after another, output feeds into the next - `hierarchical` -- a manager agent delegates tasks dynamically

---

2. Prerequisites

Requirement	Version
Python	3.10 - 3.13
pip / uv	latest
OpenAI API key (or alternative LLM)	--

Install `uv` (recommended package manager for CrewAI):

```
pip install uv
```

---

3. Repository Structure

```
my_crewai_project/
|
```

```
+-- .env                          # Secrets -- never commit
+-- .env.example                  # Template for .env
+-- .gitignore
+-- pyproject.toml                # Project metadata & dependencies
+-- README.md
|
+-- src/
|   \-- my_crewai_project/
|       +-- __init__.py
|       +-- main.py               # Entry point
|       +-- crew.py               # Crew assembly
|       |
|       +-- agents/
|       |   +-- __init__.py
|       |   +-- researcher.py
|       |   +-- analyst.py
|       |   \-- writer.py
|       |
|       +-- tasks/
|       |   +-- __init__.py
|       |   +-- research_task.py
|       |   +-- analysis_task.py
|       |   \-- writing_task.py
|       |
|       +-- tools/
|       |   +-- __init__.py
|       |   +-- search_tool.py
|       |   \-- file_tool.py
|       |
|       +-- flows/                # Optional -- advanced orchestration
|       |   +-- __init__.py
|       |   \-- pipeline_flow.py
|       |
|       \-- config/
|           +-- agents.yaml       # Declarative agent definitions
|           \-- tasks.yaml        # Declarative task definitions
|
+-- tests/
|   +-- __init__.py
|   +-- test_agents.py
|   +-- test_tasks.py
|   \-- test_tools.py
|
+-- outputs/                      # Generated artifacts
|   \-- .gitkeep
|
\-- knowledge/                    # Optional RAG knowledge base files
    \-- .gitkeep
```

Why this layout?

- `src/` layout prevents accidental imports of local source during testing
- `config/*.yaml` separates prompts/descriptions from Python logic (easier to iterate)

- `flows/` is isolated so it can be used or ignored without touching core logic

---

4. Installation & Environment Setup

4a. Scaffold with the CrewAI CLI (fastest)

```
# Install CrewAI with tools extras
pip install 'crewai[tools]'

# Scaffold a new project
crewai create crew my_crewai_project
cd my_crewai_project

# Install dependencies
crewai install
```

The CLI generates the `src/` layout, `pyproject.toml`, and starter YAML configs automatically.

4b. Manual setup

```
mkdir my_crewai_project && cd my_crewai_project
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate

pip install 'crewai[tools]' python-dotenv
```

---

5. Configuration Files

5a. `pyproject.toml`

```
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "my_crewai_project"
version = "0.1.0"
description = "CrewAI project"
requires-python = ">=3.10,<3.14"
dependencies = [
    "crewai[tools]>=0.80.0",
    "python-dotenv>=1.0.0",
]
```

```
[project.scripts]
my_crewai_project = "my_crewai_project.main:run"

[tool.crewai]
type = "crew"
```

5b. `config/agents.yaml`

```
researcher:
 role: >
   Senior Research Analyst
 goal: >
   Uncover cutting-edge developments in {topic}
 backstory: >
   You are an expert researcher with a talent for finding
   the most relevant and accurate information. You excel at
   synthesizing complex data into clear, actionable insights.

analyst:
 role: >
   Data Analyst
 goal: >
   Analyze research findings on {topic} and identify key patterns
 backstory: >
   You are a meticulous analyst who transforms raw research into
   structured, evidence-based conclusions.

writer:
 role: >
   Technical Content Writer
 goal: >
   Compose a compelling, well-structured report on {topic}
 backstory: >
   You are a skilled writer who can turn complex analyses into
   engaging, readable documents for a broad audience.
```

5c. `config/tasks.yaml`

```
research_task:
 description: >
   Research the latest developments, trends, and key facts about {topic}.
   Focus on credible sources. Provide a comprehensive summary with citations.
 expected_output: >
   A structured research summary with at least 5 key findings,
   each supported by a source reference.
 agent: researcher

analysis_task:
 description: >
   Analyze the research findings provided. Identify patterns, implications,
```

```
  and rank findings by significance. Note any conflicting information.
 expected_output: >
  An analytical report with ranked findings, identified patterns,
  and a conclusion section with strategic implications.
 agent: analyst

writing_task:
 description: >
  Using the research and analysis, write a polished final report on {topic}.
  The report must have: Executive Summary, Findings, Analysis, Recommendations.
 expected_output: >
  A full markdown report (~800 words) saved to outputs/final_report.md
 agent: writer
 output_file: outputs/final_report.md
```

5d. `.env.example`

```
# Copy to .env and fill in real values
OPENAI_API_KEY=sk-...
OPENAI_MODEL_NAME=gpt-4o

# Optional: alternative LLMs
# ANTHROPIC_API_KEY=...
# GROQ_API_KEY=...

# Optional: observability
# AGENTOPS_API_KEY=...
```

5e. `.gitignore`

```
.env
.venv/
__pycache__/
*.pyc
outputs/*.md
outputs/*.txt
outputs/*.json
.crewai/
```

---

6. Core Concepts

Agent anatomy

```
from crewai import Agent

agent = Agent(
```

```
    role="Senior Research Analyst",
    goal="Find accurate information about {topic}",
    backstory="Expert with 10 years of research experience...",
    tools=[search_tool],          # list of Tool instances
    llm="gpt-4o",                 # model string or LLM object
    verbose=True,                 # print reasoning steps
    memory=True,                  # enable short-term memory
    max_iter=10,                  # max reasoning iterations
    allow_delegation=False,       # can this agent delegate to others?
)
```

Task anatomy

```
from crewai import Task

task = Task(
    description="Research the latest AI trends...",
    expected_output="A structured list of 5 findings",
    agent=researcher_agent,       # assigned agent
    context=[previous_task],      # tasks whose output this task needs
    output_file="outputs/out.md", # optional: write result to file
    human_input=False,            # pause and ask human before finishing
)
```

Crew anatomy

```
from crewai import Crew, Process

crew = Crew(
    agents=[researcher, analyst, writer],
    tasks=[research_task, analysis_task, writing_task],
    process=Process.sequential,   # or Process.hierarchical
    verbose=True,
    memory=True,
    max_rpm=10,                   # rate limit for LLM calls
)

result = crew.kickoff(inputs={"topic": "AI in healthcare"})
```

---

7. Implementing Agents

`src/my_crewai_project/agents/researcher.py`

```
from crewai import Agent
from crewai.project import agent
from ..tools.search_tool import web_search_tool
```

```
def create_researcher(agents_config: dict) -> Agent:
    return Agent(
        config=agents_config["researcher"],    # pulls from agents.yaml
        tools=[web_search_tool],
        verbose=True,
    )
```

src/my_crewai_project/agents/analyst.py

```
from crewai import Agent


def create_analyst(agents_config: dict) -> Agent:
    return Agent(
        config=agents_config["analyst"],
        verbose=True,
    )
```

src/my_crewai_project/agents/writer.py

```
from crewai import Agent


def create_writer(agents_config: dict) -> Agent:
    return Agent(
        config=agents_config["writer"],
        verbose=True,
    )
```

---

8. Implementing Tasks

src/my_crewai_project/tasks/research_task.py

```
from crewai import Task


def create_research_task(tasks_config: dict, researcher) -> Task:
    return Task(
        config=tasks_config["research_task"],
        agent=researcher,
    )
```

src/my_crewai_project/tasks/analysis_task.py

```
from crewai import Task


def create_analysis_task(tasks_config: dict, analyst, research_task: Task) -> Task:
    return Task(
        config=tasks_config["analysis_task"],
        agent=analyst,
        context=[research_task],   # receives researcher's output
    )
```

src/my_crewai_project/tasks/writing_task.py

```
from crewai import Task


def create_writing_task(tasks_config: dict, writer, analysis_task: Task) -> Task:
    return Task(
        config=tasks_config["writing_task"],
        agent=writer,
        context=[analysis_task],
    )
```

---

9. Implementing Tools

9a. Using built-in CrewAI tools

```
# tools/search_tool.py
from crewai_tools import SerperDevTool, WebsiteSearchTool, FileReadTool

web_search_tool = SerperDevTool()          # requires SERPER_API_KEY in .env
website_tool = WebsiteSearchTool()
file_reader = FileReadTool()
```

9b. Building a custom tool

```
# tools/file_tool.py
from crewai.tools import BaseTool
from pydantic import BaseModel, Field
import json
import os


class FileWriteInput(BaseModel):
    filename: str = Field(description="Path to the output file")
    content: str = Field(description="Text content to write")
```

```
class FileWriteTool(BaseTool):
    name: str = "File Writer"
    description: str = (
        "Writes text content to a file. "
        "Use when you need to persist output to disk."
    )
    args_schema: type[BaseModel] = FileWriteInput

    def _run(self, filename: str, content: str) -> str:
        os.makedirs(os.path.dirname(filename), exist_ok=True)
        with open(filename, "w", encoding="utf-8") as f:
            f.write(content)
        return f"Successfully wrote {len(content)} characters to {filename}"


file_write_tool = FileWriteTool()
```

9c. Decorator-style custom tool (simple)

```
from crewai.tools import tool


@tool("Calculator")
def calculator_tool(expression: str) -> str:
    """Evaluates a mathematical expression safely. Input must be a valid math expression string."""
    try:
        # Only allow safe math characters
        allowed = set("0123456789+-*/(). ")
        if not all(c in allowed for c in expression):
            return "Error: invalid characters in expression"
        return str(eval(expression))  # nosec: validated above
    except Exception as e:
        return f"Error: {e}"
```

---

10. Assembling the Crew

src/my_crewai_project/crew.py

```
from crewai import Crew, Process
from crewai.project import CrewBase, crew, agent, task
import yaml
from pathlib import Path


def _load_config(filename: str) -> dict:
    config_path = Path(__file__).parent / "config" / filename
    with open(config_path, encoding="utf-8") as f:
```

```
        return yaml.safe_load(f)


@CrewBase
class MyCrewAIProject:
    """Orchestrates the full research -> analysis -> writing pipeline."""

    agents_config = "config/agents.yaml"
    tasks_config = "config/tasks.yaml"

    # -- Agents ---------------------------------------------------------

    @agent
    def researcher(self) -> "Agent":
        from .agents.researcher import create_researcher
        return create_researcher(self._agents_config)

    @agent
    def analyst(self) -> "Agent":
        from .agents.analyst import create_analyst
        return create_analyst(self._agents_config)

    @agent
    def writer(self) -> "Agent":
        from .agents.writer import create_writer
        return create_writer(self._agents_config)

    # -- Tasks ----------------------------------------------------------

    @task
    def research_task(self) -> "Task":
        from .tasks.research_task import create_research_task
        return create_research_task(self._tasks_config, self.researcher())

    @task
    def analysis_task(self) -> "Task":
        from .tasks.analysis_task import create_analysis_task
        return create_analysis_task(
            self._tasks_config, self.analyst(), self.research_task()
        )

    @task
    def writing_task(self) -> "Task":
        from .tasks.writing_task import create_writing_task
        return create_writing_task(
            self._tasks_config, self.writer(), self.analysis_task()
        )

    # -- Crew -----------------------------------------------------------

    @crew
    def crew(self) -> Crew:
        return Crew(
            agents=self.agents,    # populated automatically by @agent decorators
```

```
            tasks=self.tasks,      # populated automatically by @task decorators
            process=Process.sequential,
            verbose=True,
            memory=True,
        )
```

> ***Simpler alternative*** *-- if you do not use* `@CrewBase`*, assemble everything inline in* `crew.py` *without decorators. The decorator approach is preferred for larger projects because it enables automatic agent/task discovery.*

---

11. Entry Point & CLI

`src/my_crewai_project/main.py`

```
#!/usr/bin/env python3
"""
Entry point for the CrewAI project.

Usage:
    python -m my_crewai_project.main
    crewai run
"""

import sys
from dotenv import load_dotenv
from .crew import MyCrewAIProject

load_dotenv()  # load .env before anything else


def run():
    """Run the crew with a default topic."""
    topic = " ".join(sys.argv[1:]) if len(sys.argv) > 1 else "Generative AI in 2025"

    print(f"\n{'='*60}")
    print(f"  Starting Crew for topic: {topic}")
    print(f"{'='*60}\n")

    result = MyCrewAIProject().crew().kickoff(inputs={"topic": topic})

    print(f"\n{'='*60}")
    print("  Crew Finished")
    print(f"{'='*60}\n")
    print(result)


def train():
    """Train the crew for N iterations (improves task performance)."""
```

```
    n_iterations = int(sys.argv[1]) if len(sys.argv) > 1 else 3
    filename = sys.argv[2] if len(sys.argv) > 2 else "training_data.pkl"
    MyCrewAIProject().crew().train(
        n_iterations=n_iterations,
        filename=filename,
        inputs={"topic": "Generative AI in 2025"},
    )


def replay():
    """Replay a specific task by its ID."""
    task_id = sys.argv[1]
    MyCrewAIProject().crew().replay(task_id=task_id)


def test():
    """Run evals against the crew."""
    n_iterations = int(sys.argv[1]) if len(sys.argv) > 1 else 3
    openai_model = sys.argv[2] if len(sys.argv) > 2 else "gpt-4o-mini"
    MyCrewAIProject().crew().test(
        n_iterations=n_iterations,
        openai_model_name=openai_model,
        inputs={"topic": "Generative AI in 2025"},
    )


if __name__ == "__main__":
    run()
```

Running the project

```
# Via CLI (uses pyproject.toml script entry)
crewai run

# Or directly
python -m my_crewai_project.main "quantum computing breakthroughs"

# Train for better outputs (RLHF-style feedback loop)
crewai train 5 training_data.pkl

# Replay a specific task
crewai replay <task_id>
```

---

12. Environment Variables

Variable	Purpose	Required

`OPENAI_API_KEY`	LLM provider	Yes (default)
`OPENAI_MODEL_NAME`	Override default model	No
`SERPER_API_KEY`	SerperDev web search tool	If using SerperDevTool
`ANTHROPIC_API_KEY`	Use Claude instead of GPT	If using Claude
`GROQ_API_KEY`	Use Groq for fast inference	If using Groq
`AGENTOPS_API_KEY`	Observability via AgentOps	No
`CREWAI_STORAGE_DIR`	Override storage path	No

Using a non-OpenAI LLM

```
# Anthropic Claude
from langchain_anthropic import ChatAnthropic

llm = ChatAnthropic(model="claude-3-5-sonnet-20241022")
agent = Agent(role="...", goal="...", backstory="...", llm=llm)

# Groq (fast inference)
from langchain_groq import ChatGroq

llm = ChatGroq(model="llama-3.3-70b-versatile")

# Ollama (local)
from langchain_ollama import ChatOllama

llm = ChatOllama(model="llama3.2", base_url="http://localhost:11434")
```

---

13. Flows (Advanced Orchestration)

Flows let you compose multiple Crews into a state-driven pipeline using event-driven routing.

`src/my_crewai_project/flows/pipeline_flow.py`

```
from crewai.flow.flow import Flow, listen, start, router
from pydantic import BaseModel
from ..crew import MyCrewAIProject


class PipelineState(BaseModel):
```

```
    topic: str = ""
    research_done: bool = False
    final_report: str = ""


class ResearchPipelineFlow(Flow[PipelineState]):

    @start()
    def initialize(self):
        print(f"Starting pipeline for: {self.state.topic}")

    @listen(initialize)
    def run_research_crew(self):
        result = MyCrewAIProject().crew().kickoff(
            inputs={"topic": self.state.topic}
        )
        self.state.final_report = str(result)
        self.state.research_done = True

    @router(run_research_crew)
    def check_quality(self):
        # Route based on output quality
        if len(self.state.final_report) > 500:
            return "success"
        return "retry"

    @listen("success")
    def save_result(self):
        with open("outputs/pipeline_result.md", "w") as f:
            f.write(self.state.final_report)
        print("Pipeline complete. Report saved.")

    @listen("retry")
    def handle_retry(self):
        print("Output too short -- retrying with more context...")
        self.run_research_crew()


def run_flow(topic: str):
    flow = ResearchPipelineFlow()
    flow.kickoff(inputs={"topic": topic})
```

---

14. Memory & Knowledge

Memory types

Type	Scope	Stored in

Short-term	Within a single crew run	In-memory (default: ChromaDB)
Long-term	Persists across runs	SQLite (`.crewai/`)
Entity	Tracks named entities	SQLite
User	Per-user personalization	SQLite

```
# Enable all memory in Crew
crew = Crew(
    agents=[...],
    tasks=[...],
    memory=True,                    # enables short + long-term + entity
    memory_config={
        "provider": "mem0",         # alternative: "rag" (default ChromaDB)
    },
)
```

Knowledge (RAG over documents)

```
from crewai.knowledge.source.pdf_knowledge_source import PDFKnowledgeSource
from crewai.knowledge.source.text_file_knowledge_source import TextFileKnowledgeSource

pdf_source = PDFKnowledgeSource(file_paths=["knowledge/manual.pdf"])
text_source = TextFileKnowledgeSource(file_paths=["knowledge/faq.txt"])

crew = Crew(
    agents=[...],
    tasks=[...],
    knowledge_sources=[pdf_source, text_source],
)
```

Place source files inside the `knowledge/` directory at the project root.

---

15. Testing

`tests/test_tools.py`

```
import pytest
from src.my_crewai_project.tools.file_tool import FileWriteTool
import os


```

```
def test_file_write_tool_creates_file(tmp_path):
    tool = FileWriteTool()
    output_file = str(tmp_path / "test_output.md")
    result = tool._run(filename=output_file, content="Hello World")
    assert os.path.exists(output_file)
    assert "Successfully wrote" in result
    with open(output_file) as f:
        assert f.read() == "Hello World"
```

`tests/test_agents.py`

```
import pytest
from unittest.mock import patch, MagicMock


def test_researcher_agent_config():
    with patch("src.my_crewai_project.agents.researcher.Agent") as MockAgent:
        MockAgent.return_value = MagicMock()
        from src.my_crewai_project.agents.researcher import create_researcher
        config = {
            "researcher": {
                "role": "Senior Research Analyst",
                "goal": "Find info",
                "backstory": "Expert",
            }
        }
        agent = create_researcher(config)
        MockAgent.assert_called_once()
```

Running tests

```
pytest tests/ -v --tb=short
```

---

16. Logging & Observability

Built-in verbose logging

```
# Crew-level
crew = Crew(..., verbose=True)

# Agent-level
agent = Agent(..., verbose=True)
```

AgentOps integration

```
pip install agentops
```

```
# main.py -- add before anything else
import agentops
agentops.init(os.environ["AGENTOPS_API_KEY"])
```

Langfuse / LangSmith tracing

```
pip install langfuse   # or langsmith
```

```
# Langfuse
os.environ["LANGFUSE_PUBLIC_KEY"] = "..."
os.environ["LANGFUSE_SECRET_KEY"] = "..."
os.environ["LANGFUSE_HOST"] = "https://cloud.langfuse.com"

# LangSmith
os.environ["LANGCHAIN_TRACING_V2"] = "true"
os.environ["LANGCHAIN_API_KEY"] = "..."
```

---

17. Deployment Patterns

Docker

```
# Dockerfile
FROM python:3.12-slim

WORKDIR /app

COPY pyproject.toml .
COPY src/ src/
COPY knowledge/ knowledge/

RUN pip install --no-cache-dir 'crewai[tools]' hatchling
RUN pip install --no-cache-dir -e .

COPY .env .env

CMD ["python", "-m", "my_crewai_project.main"]
```

```
docker build -t my-crewai-project .
docker run --env-file .env my-crewai-project "AI in healthcare"
```

FastAPI wrapper (serve as an API)

```
# api.py
from fastapi import FastAPI, BackgroundTasks
from pydantic import BaseModel
from dotenv import load_dotenv
from src.my_crewai_project.crew import MyCrewAIProject

load_dotenv()
app = FastAPI(title="CrewAI API")


class RunRequest(BaseModel):
    topic: str


class RunResponse(BaseModel):
    status: str
    result: str | None = None


@app.post("/run", response_model=RunResponse)
def run_crew(req: RunRequest):
    result = MyCrewAIProject().crew().kickoff(inputs={"topic": req.topic})
    return RunResponse(status="completed", result=str(result))
```

```
pip install fastapi uvicorn
uvicorn api:app --reload
```

---

18. Full Project Walkthrough Example

Below is a condensed, self-contained single-file version useful for quick prototyping before splitting into the full structure above.

```
# quickstart.py
import os
from dotenv import load_dotenv
from crewai import Agent, Task, Crew, Process
from crewai_tools import SerperDevTool

load_dotenv()

# -- Tools ---------------------------------------------------------------
search_tool = SerperDevTool()

# -- Agents --------------------------------------------------------------
researcher = Agent(
    role="Senior Research Analyst",
    goal="Find accurate, up-to-date information about {topic}",
    backstory=(
```

```
        "You are an expert researcher skilled at finding and "
        "synthesizing information from diverse sources."
    ),
    tools=[search_tool],
    verbose=True,
)

writer = Agent(
    role="Technical Writer",
    goal="Write a clear, engaging report on {topic}",
    backstory=(
        "You transform complex research into accessible, "
        "well-structured written reports."
    ),
    verbose=True,
)

# -- Tasks ---------------------------------------------------------------
research_task = Task(
    description=(
        "Research the latest developments about {topic}. "
        "Find at least 5 credible facts with sources."
    ),
    expected_output="A structured list of 5+ findings with citations.",
    agent=researcher,
)

writing_task = Task(
    description=(
        "Using the research findings, write a 500-word report "
        "on {topic} with sections: Summary, Key Findings, Conclusion."
    ),
    expected_output="A polished 500-word markdown report.",
    agent=writer,
    context=[research_task],
    output_file="outputs/report.md",
)

# -- Crew ----------------------------------------------------------------
crew = Crew(
    agents=[researcher, writer],
    tasks=[research_task, writing_task],
    process=Process.sequential,
    verbose=True,
)

# -- Run -----------------------------------------------------------------
if __name__ == "__main__":
    result = crew.kickoff(inputs={"topic": "AI agents in 2025"})
    print("\n--- FINAL OUTPUT ---")
    print(result)
```

```
pip install 'crewai[tools]' python-dotenv
python quickstart.py
```

Quick Reference Cheat Sheet

```
crewai create crew <name>    -> scaffold new project
crewai install               -> install deps from pyproject.toml
crewai run                   -> run the crew
crewai train <n> <file>      -> train for n iterations
crewai replay <task_id>      -> replay a specific task
crewai test <n> <model>      -> evaluate crew outputs
```

Decorator	Used in `@CrewBase` class	Purpose
`@agent`	Method returning `Agent`	Auto-register agent
`@task`	Method returning `Task`	Auto-register task
`@crew`	Method returning `Crew`	Define crew assembly
`@tool`	Standalone function	Define simple tool
`@start`	Flow method	Flow entry point
`@listen(x)`	Flow method	Trigger after step x
`@router(x)`	Flow method	Conditional routing

Generated: 2025 -- based on CrewAI v0.80+